

ADR-0001 — LogLine-native digestion of contracts, norms, and specs

Purpose

Freeze, in one authoritative place, what we understand about LogLine and the Lab — from the Foundation canon to the external-standards research — and to correct an ontological error in the first draft of this record, so that both the existing and future implementation get better and the real bootstrap becomes *assembly, not invention*.

This document is, deliberately, only a seed and a projection of itself: the rough human material an LLM digests into one or more LogLine Acts, and the human-readable rendering those Acts can later project back into. The truth is the registered Act, not this file.

Scope and companion records

This record governs **what a thing is and how it comes to be** — digestion, the Act, the envelope, custody, and the registration-vs-activation law. It deliberately does **not** govern how things *move* or how the Lab *grows*; that is its companion:

- **ADR-0002 — Process, risk, and governed autonomy:** process-not-source-privilege, the risk matrix, scale, automation, jurisprudence, autonomy, and the enzyme-authority boundary.
- **ADR-0003 — Projections & Attention (v4/v5):** the eyes and the governed presence.

Read together they are complementary: **ADR-0001 = how a thing comes to be** · **ADR-0002 = how it is treated and moves** · **ADR-0003 = how it is seen and what earns attention.**

Correction being made

The first draft framed the Lab as if the document were the noble body that later “receives an envelope” or “becomes a receipt” — *human-readable body over machine-readable envelope*. That is backwards.

The Lab is not document-native. The Lab is LogLine-native. Markdown is either pre-LogLine rough material or post-LogLine projection. The native object is the LogLine Act: human material *digested* into structured processual form. The processual envelope comes *after* the Act, to situate it for routing, activation, transport, attention, supersession, or consequence. The registered receipt is custody; the Markdown document is a projection.

This ADR therefore supersedes three concepts (see `aux.supersedes_concepts`): `markdown-as-canonical-body`, `document-first-receipt-later`, and

body-over-envelope-as-primary-model.

Context

The Lab is Dan’s private implementation of the LogLine method (the LogLine Foundation is separate; Dan created it). Everything must be **LogLine-Foundation canon in form and spirit**, processually grounded in **Universal Inbox v3/v4/v5**. The Lab is generic infrastructure to organize Dan’s life, computers, research, and businesses — not a product for others.

The epistemic motive matters and must be stated. A human needed to model and compress the chaos of daily life; even the computer could not do it — until a grammar specialist did. LogLine is a way to **freeze human intention** statically, cheaply, and simply, with **very small loss when the freezing is performed by an LLM**. The nine slots and AUX are not metadata bolted onto a document; they are *the organizing of an LLM’s thought* — the natural grammar by which language becomes act-structure. The power of LogLine lives precisely there.

A concrete gap surfaced during Day 2: process contracts live as disk files (`processes/*.v1.yml`), the rules that matter are real fields but the rationale is YAML comments (silently dropped by the parser and absent from any hash), and **nothing is digested into the ledger**. For contracts and norms to be valid and consultable in production (Supabase), they must be Acts, not files.

Native ground (what the canon already legislates)

From LAB FINAL SPEC v0.md and the LLM-facing canon:

- **The receipt is the truth; projections, columns, views are maps** (§5, :224). A file may *initiate* the digestion and a file may *render* the result, but the file is not the institutional truth.
- **Receipt discipline:** nine stable slots (`who`, `did`, `this`, `when`, `confirmed_by`, `if_ok`, `if_doubt`, `if_not`, `status`) + **AUX** + **tuple_hash** + **content_hash** + **JCS/RFC8785** canonicalization + append-only custody (§5, :228–247). Consequence may not be smuggled into forbidden top-level fields (`transport/result/evidence`); `transport` belongs to the **envelope** boundary (`envelope_hash`).
- **Composition is citation-by-hash:** “*Evidence is records or evidence objects cited by records*” (:254); evaluation asks “*Are cited hashes resolvable?*” (:291); “*Cite before claiming*” (:1561). Resolvability of cited hashes is already an activation gate.
- **Registration activation:** everything may register; only complete records matching a registered process contract activate; evaluation classifies, **does not reject** (§6).
- **A process is a contract, not a daemon** (§8, :400), carrying `...danger` tier, `grant` requirements, `evidence` obligation, `... version`,

supersession (:404–423). New capability is added by *adding/updating contracts, schemas, adapters, projections* — *not new daemons* (:433).

- **Closure states** include **fechado**, **doubted**, **ghost**, **retired**, **superseded** (§9, :460–467): supersession is a first-class lifecycle — the cadence of rules.
- **The three method layers** (not three versions of one thing):
 - **Awakening / Universal Inbox (v3)** (§11): “*Register yourself fully if you want to be woken. Address canonically if you want to be heard.*” A sender can *trigger* but never *define* a receiver’s behavior; a malformed message rings only a bell already wired; never infer effect from string shape (explicit mode → process contract → executor approval → adapter action).
 - **Projections (v4)** (§14): the Lab’s eyes — **authoritative:false**, **rebuildable:true**, declaring sources + **input_hashes** + **class**. Dynamic projections are **ladders** for LLM reasoning (L0 raw Acts → ... → L5 candidate Acts); “an LLM climbs to think and descends to verify”; every abstraction is traceable to lower-level hashes.
 - **Attention (v5)** (§15): **scarce**, **governed presence** — **doubted** feeds attention; it is *not an error queue*; it prevents the system from pretending uncertainty is failure or silently escalating it into action.

Registration vs activation (the v3 law)

This corrects an old two-phase misreading. **Everything may register.** Registration means *the Lab remembers this* — it is custody, not trust, not approval, not correctness. A registered Act is truth-as-recorded the instant it is registered.

Activation is earned by form, and it is separate. If the registration’s form matches a registered process contract (complete slots/AUX, resolvable citations, active process, authority/ grant/budget satisfied), it activates — *good*. If it does not, **it is just a register**: a remembered, inert fact. Nothing is rejected; evaluation classifies.

There is no “candidate” state in the ledger. “candidate” lives only *before* registration — as a v4 projection-ladder output or a dream-machine (imagination) proposal. The LLM may emit candidate Acts while reasoning; the moment one is registered it is a register, never a half-truth awaiting promotion. The Lab’s power is precisely this: it promotes the register of *anything*, and lets form — not a gatekeeper — decide what moves.

The wrong model

human writes a document

- > the document is the canonical body
- > later it "gets an envelope"
- > later it "becomes a receipt"

This treats Markdown as canonical content and the problem as documental representation. It makes the file the truth and the Act an afterthought. It is rejected.

The corrected model

Human material is digested into LogLine-native Acts. Markdown is either pre-LogLine rough material or post-LogLine projection. The semantic and processual content is digested into one or more LogLine Acts; any Markdown body is source material, evidence, or projection — never the canonical body.

The pipeline

```
raw human material
-> LLM digests
-> LogLine Act (structured, native)
-> processual envelope / route / activation / transport
-> receipt registered in the ledger (custody)
-> human projections, including Markdown
```

Raw human material may be: speech, draft, rough Markdown, conversation, audio, image, code, a human contract, a desire, an informal decision, an aesthetic correction, a doubt, a memory.

LLM digestion performs: interpretation, separation, naming, structuring, decomposition, intent detection, doubt preservation, process identification, AUX formation, and the proposal of one or more LogLine Acts.

What is a LogLine Act in this decision

The native, structured form — the frozen intention:

```
who
did
this
when
confirmed_by
if_ok
if_doubt
if_not
status
AUX
```

Contracts, norms, specs, grants, decisions, vocabularies, results, and evidence become Lab-native when they are digested into LogLine Acts and then registered, cited, evaluated, activated, or projected according to process. They are not “documents that get stored”; they are intentions frozen into act-structure.

What is the processual envelope

Posterior to, and distinct from, the Act. It situates an existing Act for movement and consequence:

```
process
route
contract
activation
transport
sent_to
runtime
idempotency
grants
evidence obligation
projection target
supersession target
```

The Act says *what was meant*; the envelope says *where it goes and how it may move*. The envelope never precedes the structure it carries.

What Markdown is and is not

Markdown IS: rough human material (a seed) OR a human-readable projection
Markdown IS NOT: the canonical body, the institutional truth, or the unit of custody

A readable document, a rendered spec, a human ADR, a preview, a report, an explanation, an index, a UI surface — all are projections of registered Acts, not the Acts themselves.

Role of the LLM

The LLM is not merely extracting metadata from a document; it is performing the natural LogLine digestion of language into act-structure. This is the epistemic act at the center of the method: the LLM is the low-loss freezer of human intention. It interprets, structures, names, decomposes, detects intent, and — crucially — **preserves doubt** (as `if_doubt`, attention, or an open reconciliation) rather than discarding it. The human directs, judges, corrects, and chooses format; the LLM digests, investigates, structures, warns, and proposes.

External standards as confirmation, not containers

The external standards confirm parts of the *process* and lend *vocabulary*. They do not become the container; the LogLine Act remains the native form.

- **in-toto** / **SLSA** confirms the digest/evidence/envelope vocabulary (envelope + subject by digest + typed predicate). LogLine independently has this shape; the confirmation is welcome, the import is not.

- **W3C VC / DID** confirms grant/issuer/subject/status vocabulary; a grant is an Act, not a foreign credential object.
- **Cedar / OPA** confirms authorization vocabulary (principal/action/resource/context) — the *model*, never the engine, which would never enter the zero-dependency kernel.
- **CUE / JSON Schema** confirms validation vocabulary for the Act/contract shape.
- **Structured-MADR** confirms decision/rationale vocabulary — the reason a decision Act carries its rationale as data.

Rationale is data

A comment is only a human convenience. It does not survive parsing and does not enter the canonical hash; it is invisible to the Lab. If a reason matters to the Lab it must be one of:

```
an AUX field
a cited Act
an evidence record
a norm reference
a decision reference
a projection with input hashes
```

The YAML comments added to contracts on Day 2 are, by this law, not yet institutional; they are rough material awaiting digestion into AUX or into cited norm/decision Acts.

Decision (the laws this record sets)

1. **The Lab is LogLine-native, not document-native.**
2. **Markdown is rough material before digestion or projection after digestion** — never the canonical body.
3. **LLM digestion produces LogLine Acts** — the natural freezing of language into act-structure.
4. **The LogLine Act is the native structured form** (nine slots + AUX, JCS-canonical, content-addressed).
5. **The processual envelope situates the Act after structure exists** (route, activation, transport, attention, supersession, consequence).
6. **Registered receipts provide custody/truth**; files seed or render, they do not hold truth.
7. **Composition is by hash** — contracts, authorities, sources, evidence, and superseded norms are cited by `content_hash`; resolvability is an activation gate.
8. **Supersession is append-only** — a new version is a new Act citing and superseding the prior by hash; old Acts are never edited in place.
9. **Rationale is data** — if it matters, it lives in AUX or a cited Act, never in a comment.

10. **External standards lend vocabulary only** — they confirm shapes; they are never the container.

Representation / digestion pass

The central correction, expressed as work. The order matters and each step compounds:

1. Identify the rough human materials (the contracts, notes, norms, playbooks we already wrote).
2. Ask the LLM to digest them into candidate LogLine Acts.
3. Preserve ambiguity as `if_doubt` / `attention` / `open_reconciliations` - never discard it.
4. Validate the Act shape (against the receipt/contract schema).
5. Attach or cite the source-material hash (a file initiates; the Act remembers its seed).
6. Register the Acts as receipts - everything may register. Whether an Act then `*activates*` process is decided by its form (the v3 law), not by an acceptance gate; an Act that activates nothing is still a valid register, remembered and citable.
7. Generate Markdown projections FROM the registered Acts (v4).
8. Build `spec/contract/norm` projections from the ledger - not the reverse.

Dependency-light, canon-native, and mapped onto the roadmap: **Day 4** (Projection Spec Registry + Spec Schema; projections already carry a `projection_spec_hash`), **Day 7** (Knowledge Pack / Bootstrap Book), **Day 8** (genesis registers specs; genesis already carries a `spec_hash`).

Consequences

- **Better existing implementation:** contracts gain self-describing, registerable canon; rationale stops being invisible; the danger/mold vocabularies stop being hardcoded constants and become cited vocabulary Acts.
- **Better future implementation:** the bootstrap becomes “digest the seeds into Acts and resolve by hash” — no bespoke engine to invent. Supply-chain interop (in-toto/SLSA tooling) is available for free because the shape already matches.
- **The Dream Machine UI is not a Markdown editor first.** It is a **LogLine digestion IDE**:
 - the **Ledger column** shows registered Acts;
 - the **Projections column** shows ladders and dynamic projections;
 - the **Preview column** renders human-readable output (incl. Markdown) from Acts/projections;
 - **LLM prompt boxes** are situated by column/context;
 - the **human** directs, judges, corrects, and chooses format; the **LLM** digests, investigates, structures, warns, and proposes.
- **Cost / risk:** a one-time digestion pass; a risk of over-formalizing — mitigated by keeping engines out of the kernel and treating standards as vocabulary only.

Open reconciliations

1. How much raw source material is stored inline vs. cited by hash?
2. Which **did** families are official for digested Acts (e.g. **process-contract**, **norm**, **decision**, **vocabulary**)?
3. How is source-material hash / body hash represented alongside the Act?
4. How do digested Acts surface through **Projections v4** (which ladder levels) and **Attention v5** (what becomes governed presence vs. inert register)? The three layers are now read from canon (§11/§14/§15); what remains is the v4/v5 vocabulary for *digested* Acts.
5. A digested Act is registered immediately (everything may register); a *proposal* that is not yet registered lives in imagination (dream-machine) or a projection ladder. Where exactly does the digestion IDE draw that line — what does a human/process see before choosing to register? (Note: this is NOT a candidate-promotion gate; there is no candidate state in the ledger.)
6. How are Markdown projections regenerated deterministically from Acts?
7. How are corrections/supersessions represented without editing old Acts?

Status

registered / accepted. This is the north-star for the digestion pass. Supersede only by a later **did=decision** receipt that cites this record's hash — never by edit-in-place.